

Explanation about the Diagrams

1. High-Level Architecture Diagram

Shows the end-to-end system in four layers: external **Sources** (RSS/APIs/scrapers) feed an **Ingestion Layer**, which queues articles into Kafka for the **Processing Pipeline** (normalize → dedup → cluster → rank), which writes into the **Storage Layer** (raw article store, story DB, vector index, feed cache), which the **Serving Layer** (Feed API, Search API) reads from to serve **Client Apps**. Read-heavy path is cached; write path favors correctness.

2. Ingestion Layer (idempotency flow)

Every polled article gets an idempotency key (hash(source_id + canonical_url)) checked against Redis before it's allowed onto the Kafka queue. If the key was seen recently, the article is dropped as already-ingested; otherwise the key is stored with a TTL and the article is pushed forward. This stops naive re-polling duplicates before they ever enter the processing pipeline.

- **Fetcher/Poller workers:** scheduled workers (cron-like, per-source polling interval based on source's historical publish frequency) pull RSS/API data. For high-volume publishers, prefer **webhooks/push** over polling where available.
- Each fetched article is pushed onto a **Kafka topic** (partitioned by source ID) — this decouples ingestion rate from processing rate and lets you replay/backfill if a downstream stage fails.
- **Idempotency at ingestion:** compute a fetch-dedup key (e.g., hash of source_id + canonical URL) before pushing to the queue, so re-polling the same RSS entry doesn't re-enqueue it. Use a short-TTL cache (Redis) of recently seen (source, URL) pairs — cheap and covers the vast majority of naive duplicates from re-polling.

3. Story Clustering (misabeled "Re-ranking" — relabel this)

A candidate article that survived the dedup checks is compared against nearby story centroids in the ANN index. If similarity exceeds a threshold, it's attached to that existing story (centroid updated, source_count incremented); otherwise a new story is created. Either outcome updates the Story DB and triggers downstream re-ranking, since the story's signals just changed.

4. Ranking / Scoring Service

Four independent signals are computed per story — **recency** (exponential decay by age, so older stories fade), **source reliability** (a pre-assigned trust weight per publisher), **relevance** (match against a user's interests or search query), and **source count** (how many independent sources are covering it, used as a popularity proxy). These four feed into a **weighted combination** step (a simple weighted sum to start, with room to swap in a learning-to-rank model later without changing the rest of the pipeline). The combination produces a single **final story score**, which is then **written to the Feed Cache** — so by the time a user actually requests their feed, the score is already computed and sitting in cache; nothing gets recalculated on the read path.